

Universidade Federal do Ceará — Bacharelado em Sistemas e Mídias Digitais

SMD0033 — Estrutura de Dados Prof. Ernesto Trajano de Lima

Mini-Projeto

Catálogo de fotos indexado por timestamp (fotolog)

1. Contexto

Um *fotolog* é um registro pessoal organizado de fotografias, em que cada imagem aparece acompanhada de metadados — data do disparo, local, descrição, etiquetas, avaliação. A operação central do usuário é navegar pelo acervo em ordem cronológica: visualizar um intervalo de datas, pular para a foto mais próxima de um instante, avançar e retroceder pelo tempo.

Neste mini-projeto, cada aluno implementará o núcleo de um *fotolog* cujo índice principal é uma árvore binária de busca (BST), com o timestamp da foto como chave. As consultas centrais (intervalo, vizinho mais próximo, sucessor/predecessor cronológico) devem ser resolvidas explorando a propriedade da BST, não percorrendo o acervo inteiro.

2. Objetivos de aprendizagem

- Aplicar a BST estudada em sala (material “arvores_I.pdf” e “arvores_II.pdf”) num cenário não-trivial, com chave composta e desempate determinístico.
- Implementar corretamente as rotinas que costumam ser glosadas em aula: sucessor e predecessor in-order com subida pelo pai, e remoção com nó de dois filhos.
- Reconhecer quando uma travessia ingênua é desnecessária — projetar uma consulta de intervalo que poda sub-árvores fora do intervalo (custo $O(\log n + k)$).
- Combinar uma estrutura de árvore com um índice secundário auxiliar (dict por id), articulando os dois sem que um substitua o outro.
- Submeter o resultado a uma bateria de testes que exercite carga em massa, casos limítrofes e operações mistas.

3. Especificação funcional

O *fotolog* deve oferecer, no mínimo, as operações abaixo.

3.1. Modelo de dado

Cada foto tem id (inteiro único), ts (timestamp em segundos), path (string), tags (lista de strings) e rating (inteiro em $[0, 5]$). Duas fotos podem ter o mesmo ts (rajada de disparos), mas nunca o mesmo id.

3.2. Operações do catálogo

Operação	Comportamento esperado
<code>add(foto)</code>	Insere uma foto. Rejeita id duplicado.
<code>remove(id)</code>	Remove a foto pelo id. Sinaliza erro se não existir.
<code>get_by_id(id)</code>	Recupera a foto pelo id.
<code>range(ts1, ts2)</code>	Lista, em ordem cronológica, todas as fotos com ts em [ts1, ts2].
<code>nearest(ts)</code>	Devolve a foto com timestamp mais próximo de ts.
<code>next_of(id) / prev_of(id)</code>	Sucessor / predecessor cronológico da foto com aquele id.
<code>remove_range(ts1, ts2)</code>	Remove em lote todas as fotos no intervalo, mantendo a consistência da árvore e do índice secundário.
<code>tag(id, t) / rate(id, r)</code>	Edita metadados.
<code>find_by_tag(tag)</code>	Lista, em ordem cronológica, as fotos com aquela tag.
<code>stats()</code>	Total, mais antiga, mais recente, rating médio e mediano.
<code>save(path) / load(path)</code>	Persistência em JSON.

3.3. Interface

Um loop de comandos prefixados por “:”, conforme a tabela abaixo. Os comandos devem ser robustos a erros de uso (mensagem clara, sem *stack trace*).

Comando	Efeito
<code>:add <id> <ts> <path> [rating]</code>	Adiciona uma foto manualmente.
<code>:import <manifest.json></code>	Importa várias fotos de um JSON; registros inválidos são ignorados e o total é reportado.
<code>:range <ts1> <ts2></code>	Lista fotos com ts em [ts1, ts2].
<code>:nearest <ts></code>	Foto com timestamp mais próximo.
<code>:next <id> / :prev <id></code>	Navegação cronológica.
<code>:get <id></code>	Mostra os metadados de uma foto.
<code>:tag <id> <tag></code>	Anota uma tag.
<code>:rate <id> <0..5></code>	Atribui rating.
<code>:find-tag <tag></code>	Lista fotos com a tag (in-order).
<code>:remove <id> / :remove-range <ts1> <ts2></code>	Remoção pontual / em intervalo.
<code>:stats / :list / :tree</code>	Estatísticas / listagem cronológica / ASCII da árvore.

Comando	Efeito
:save <f> / :load <f>	Persistência.
:help / :quit	Ajuda / encerrar.

O parâmetro ts deve aceitar inteiro (segundos desde a epoch) ou string ISO: YYYY-MM-DD, YYYY-MM-DD HH:MM ou YYYY-MM-DD HH:MM:SS.

3.4. Manifest JSON

Lista de objetos contendo os campos da foto. Exemplo:

```
[
  {"id": 1, "ts": 1704067200, "path": "/p/1.jpg",
   "tags": ["praia"], "rating": 5},
  {"id": 2, "ts": "2024-03-22 16:05:30", "path": "/p/2.jpg"}
]
```

4. Estrutura de dados central

A estrutura primária é uma BST, que pode ser subclasse da BinaryTree disponibilizada no SIGAA ou utilizar uma BST como atributo privado. A chave de ordenação é o par (ts, id) — o id atua como desempate determinístico quando dois ts coincidem, garantindo um in-order estritamente crescente.

É permitido (e recomendado) manter um índice secundário auxiliar, um dict por id, para que :get e :tag tenham acesso $O(1)$ à foto sem percorrer a árvore. O índice secundário é apenas um atalho: todas as consultas com semântica cronológica (range, nearest, next/prev) devem ser resolvidas pela BST.

Restrição. Você não pode substituir a BST por uma lista ordenada, por um dict ou por qualquer outra estrutura que dispense a árvore. O objetivo é exercitar a BST.

5. Testes

Submeta sua implementação a uma bateria de testes. Ela deve cobrir, no mínimo:

- Carga em massa de pelo menos 1000 inserções aleatórias, verificando que o in-order resultante é estritamente crescente.
- range em janelas aleatórias, comparado contra um baseline produzido por filtro+sort do mesmo conjunto.
- nearest cobrindo todos os casos limítrofes: antes da primeira foto, depois da última, empate exato, e equidistante.
- remove_range seguido de uma verificação de invariantes — tamanho da árvore == tamanho do índice, in-order ordenado, conjuntos de ids idênticos.
- Sucessor de um nó sem filho direito, projetado para forçar a subida pelo pai (testa explicitamente a correção do item 5).

CrITÉRIOS adicionais (deleção em todos os trÊs casos, persistência round-trip, REPL) somam pontos no item de qualidade.

7. Organização

A divisão em módulos abaixo descrita é uma sugestão razoável; outras divisões equivalentes podem ser utilizadas desde que a leitura do código continue clara.

Arquivo	Conteúdo
binary_tree.py	O arquivo do material, sem modificações.
photo.py	Classe Photo com ordenação por (ts, id) e parsers de timestamp.
photo_bst.py	Subclasse de BinaryTree com insert, find, range, nearest, sucessor/predecessor e delete.
catalog.py	Camada de catálogo: BST + índice secundário + persistência.
repl.py	REPL de comandos.
test_fotolog.py	Bateria de testes unitários.
README.md	Como executar, decisões de projeto, limitações conhecidas.

9. Observações

- Pode-se usar apenas a biblioteca padrão do Python (módulos como json, statistics, shlex, dataclasses, datetime são permitidos).
- Não use bibliotecas externas de árvore (sortedcontainers, bintrees, etc.) — a estrutura tem de ser sua.